# Campus AI Platform

## Technical and Roadmap Document

*Version 2.0 | 24 October 2026*

Open-source, role-based, multimodal AI assistant for universities. Built with Streamlit and Gemma 4. Prepared for Infinity Hacks 2.0 (26 October 2026).

***Abstract.*** *The Campus AI Platform is a single Streamlit application in which students, faculty and administrators see only the modules their role allows. It combines a grounded retrieval-augmented knowledge assistant with conversation memory, multimodal explanation and diagram review, faculty tools for marking support, outcome mapping and clash-free timetables with labs, and student tools for career roadmaps, voice interview practice, placement odds and project discovery. All behaviour that differs between universities lives in YAML files.*

*Live URL:: [https://vitapcampusai.streamlit.app/](https://vitapcampusai.streamlit.app/)*

## 1. Introduction and Scope

Universities hold their rules, calendars and procedures in scattered documents, and faculty spend large amounts of time on repetitive academic paperwork. This platform provides one open-source application that any institution can deploy and adapt by editing configuration, with no code changes.

### Design principles

- **Config over code:** rubrics, outcomes, roadmaps, company history, interview questions and prompts are YAML files.
- **Plug-in modules:** a feature is one file with a `MANIFEST` (name, roles, order) and a `render(user)` function; the menu is built from the manifests.
- **Human in the loop:** the AI suggests marks, tags and scores; faculty approve before anything is saved as final.
- **Grounded answers:** the knowledge assistant answers only from uploaded documents and refuses otherwise.
- **Graceful degradation:** with no API key the knowledge assistant (keyword mode), CO-PO mapper, timetable, roadmap, placement model and diagram rule checks still work.
- **Open and portable:** runs on Streamlit Community Cloud with the Gemini API, or locally with SQLite and Ollama (open weights).

**TABLE I**
FEATURE INVENTORY AND STATUS

| Feature | Role | Status |
|---|---|---|
| Campus Knowledge Assistant (RAG) | All | Implemented |
| Conversation memory (follow-ups, saved history) | All | Implemented (new) |
| Snap-and-Explain (English/Telugu) | Student, Faculty | Implemented |
| Diagram Reviewer (model extraction + rule checks) | Student, Faculty | Implemented |
| Answer-Script Feedback Assistant | Faculty | Implemented |
| CO-PO Mapper and Question Paper Auditor | Faculty | Implemented |
| Timetable Generator with labs and share link | Faculty, Admin | Implemented (new) |
| Career Roadmap for a selected goal | Student | Implemented (new) |
| Interview Mock Bot (voice, scoring) | Student | Implemented (new) |

| Feature | Role | Status |
|---|---|---|
| Placement Probability (Bayesian) | Student | Implemented (new) |
| Project Showcase (auto-tags, similar, teammates) | All | Implemented (new) |
| Knowledge Base, Config, User, Usage/Audit/Health | Admin | Implemented |

## 2. Roles and Access Matrix

**TABLE II**
MODULE ACCESS BY ROLE

| Module | Student | Faculty | Admin |
|---|---|---|---|
| Campus Knowledge Assistant | Use | Use | Use |
| Project Showcase | Use | Use | Use, delete any |
| Snap-and-Explain | Use | Use | - |
| Diagram Reviewer | Use | Use | - |
| My Career Roadmap | Use | - | - |
| Interview Mock Bot | Use | - | - |
| Placement Probability | Use | - | - |
| Timetable Generator | - | Use | Use |
| Answer-Script Feedback | - | Use | - |
| CO-PO Mapper and Paper Auditor | - | Use | - |
| Knowledge Base Manager | - | - | Use |
| Config Manager | - | Read only | Edit |
| User Management | - | - | Use |
| Usage, Audit and Health | - | - | Use |

Documents carry an audience (all, students, faculty, admin). The role filter is applied inside the database query before any scoring, so material meant for faculty can never be retrieved by a student. A shared timetable link is the one public, read-only surface.

## 3. System Architecture

A layered logical view (Fig. 1), an information-flow view of the knowledge pipeline (Fig. 2) and a deployment view (Fig. 3).

**Layers.** Users reach a single Streamlit application over HTTPS. The presentation layer shows the login page or, for a `?tt=CODE` URL, the public timetable view. The application layer holds the plug-in modules, grouped by the role that may open them. Core services provide authentication, the module registry, configuration loading, audit logging, the RAG engine, chat memory, the LLM client and the rule engines. State is kept in one relational database (SQLite by default, PostgreSQL for persistence) and in YAML files. Model calls go to the Gemini API (Gemma 4 for text and vision, an embedding model, a speech model) or to a local Ollama server.

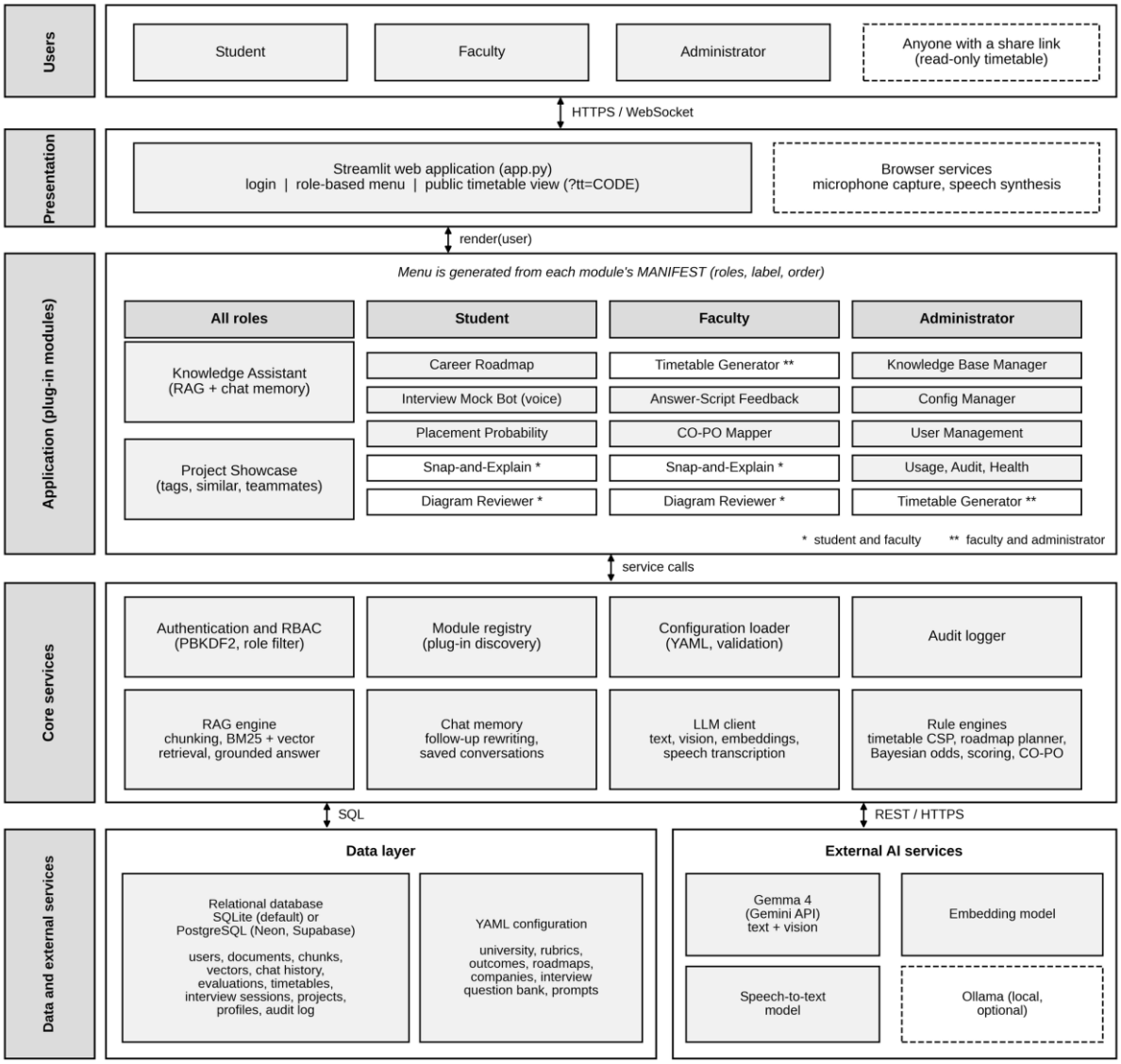



**Fig. 1.** Layered architecture of the Campus AI Platform.

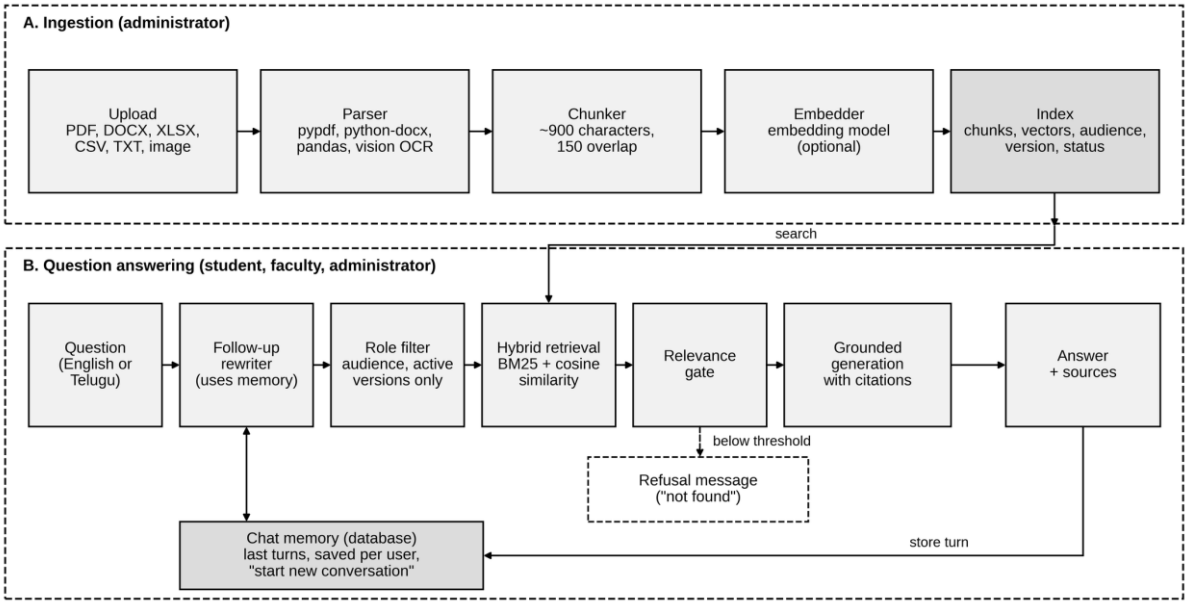



**Fig. 2.** Knowledge pipeline: document ingestion and conversation-aware question answering.

**Knowledge pipeline.** Administrators upload documents, which are parsed, split into chunks of about 900 characters with 150 overlap, optionally embedded, and indexed with audience, version and status. A question is first rewritten into a standalone form using the conversation memory, filtered by the asker's role, scored by BM25 plus cosine similarity, and passed through a relevance gate. Only passages that pass the gate reach the model, which must answer from them with citations; otherwise the user receives a not-found message.

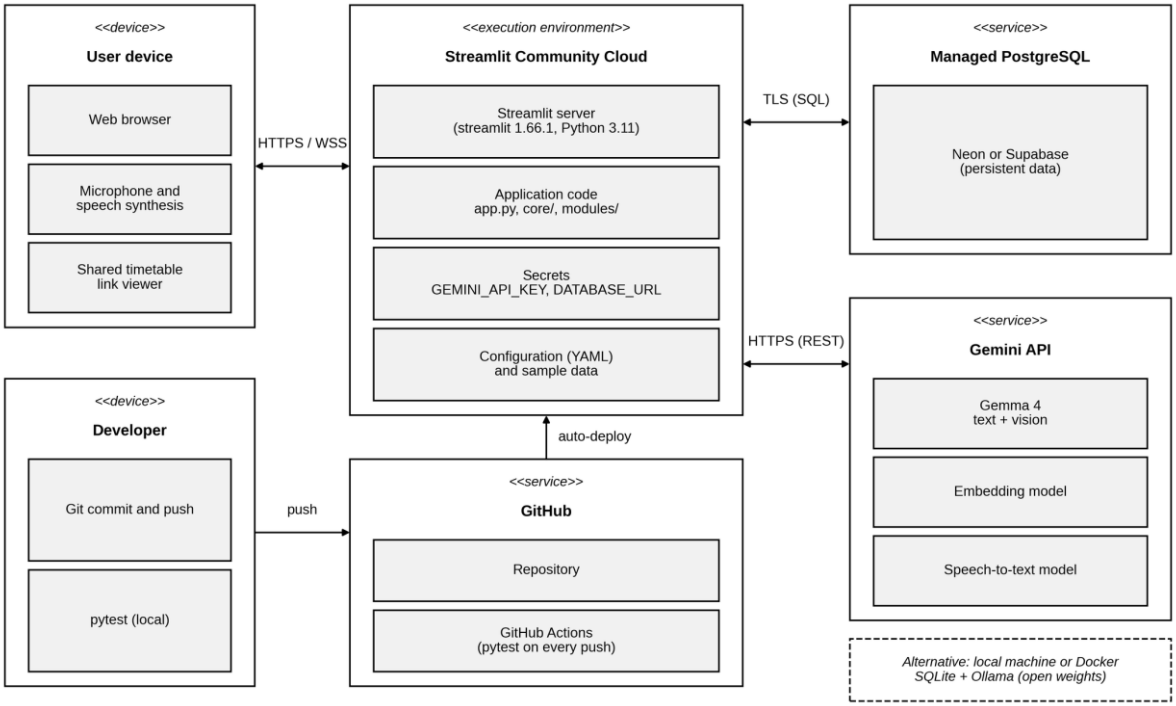



**Fig. 3.** Deployment view: Streamlit Community Cloud with managed PostgreSQL and the Gemini API.

**Deployment.** The public demo runs on Streamlit Community Cloud, built from GitHub on every push. Secrets (`GEMINI_API_KEY`, `DATABASE_URL`) are held in the platform's secret store. A free managed PostgreSQL service (Neon or Supabase) makes uploaded data persistent; without it SQLite is used and resets when the app restarts.

## 4. Technology Stack

**TABLE III**
TECHNOLOGY CHOICES

| Layer | Choice | Reason |
|---|---|---|
| Web framework | Streamlit 1.66.1 (pinned) | Fast to build; deploys free on Streamlit Cloud; AppTest enables headless UI tests |
| Language model | Gemma 4 via Gemini API (default `gemma-4-26b-a4b-it`); optional Ollama | Open-weights family with vision; local option for privacy |
| Embeddings | `gemini-embedding-001` (configurable) | Multilingual, supports Telugu retrieval |
| Speech to text | Audio-capable Gemini model (`TRANSCRIBE_MODEL`) | Voice answers in the interview bot |
| Speech output | Browser SpeechSynthesis | No server cost; works offline from the model |
| Database | SQLAlchemy over SQLite or PostgreSQL | One connection string switches engines |
| Retrieval | BM25 (own code) + NumPy cosine | No heavy vector database needed at department scale |

| Layer | Choice | Reason |
|---|---|---|
| Parsing | pypdf, python-docx, pandas/openpyxl, vision OCR | PDF, DOCX, XLSX, CSV, TXT, images |
| Config and prompts | YAML | Editable without code changes |
| Tests and CI | pytest, Streamlit AppTest, GitHub Actions | 88 automated tests run on every push |

## 5. Repository Structure

```
campus-ai/
  app.py                    entry point: login, role menu, public timetable route
  requirements.txt          pinned runtime dependencies
  config/                   university, rubrics, outcomes, roadmaps, placement,
                            interview question bank, prompts
  core/                     auth, db, llm, rag, chat_memory, parsing, rubric, copo,
                            diagram_checks, timetable, roadmap, placement, interview,
                            projects, registry, seed, audit, config
  modules/                  one file per feature (MANIFEST + render)
  data/sample_docs/         fictional demo documents (loaded on first run)
  data/test_data/           answer scripts, diagrams, papers, projects, users
  scripts/                  eval_rag.py, generate_test_data.py, reset_db.py
  tests/                    88 automated tests
```

## 6. Core Services

### *6.1 Authentication and access control*

Passwords are stored as salted PBKDF2-SHA256 hashes. A login creates a session user with a role (student, faculty, admin). The menu is built from the module manifests for that role, and role filters are applied inside data queries. Three demo accounts are seeded on first run; a real deployment should hide them and change the passwords.

### *6.2 LLM client*

One interface exposes `generate`, `generate_json`, `embed` and `transcribe`. Backends are the Gemini API and Ollama. System instructions are placed inside the prompt because Gemma models on the Gemini API do not take a separate system field. Rate-limit and availability errors are retried with exponential backoff. JSON requests are repaired once by re-asking with a stricter instruction before failing. If no key is configured the client reports unavailable and every caller falls back to rule-based behaviour.

### *6.3 Configuration and prompt packs*

Settings come from environment variables, then a local `.env`, then Streamlit secrets. Behaviour tables live in YAML under `config/`. The admin Config Manager validates rubric mark sums and CO-PO references before saving.

### *6.4 Module registry*

Each file in `modules/` defines `MANIFEST = {id, label, icon, roles, order}` and `render(user)`. The registry imports them, sorts by order and filters by role, so adding a feature never touches the menu code.

### *6.5 Audit logging*

Every AI-assisted action records user, role, module, action, model and time. The admin dashboard shows and exports the log.

# 7. Module Specifications

### *7.1 Campus Knowledge Assistant with conversation memory*

**Purpose.** Students and staff ask questions in English or Telugu and receive cited answers drawn only from documents the administrator uploaded.

- **Ingestion:** upload, parse, chunk (about 900 characters, 150 overlap), embed when a key is available, store with title, category, audience, version, effective date and status. A new upload with the same title archives older versions.
- **Retrieval:** role filter in SQL, then score = 0.4 x normalised BM25 + 0.6 x cosine when vectors exist, otherwise BM25 alone.
- **Relevance gate:** the question is refused when the best cosine is below 0.55 and fewer than 34 percent of its terms appear in the best passage. Both thresholds are configurable.
- **Generation:** the model must answer only from the numbered passages, cite them, prefer the latest effective date when passages conflict, treat passage text as untrusted data, and reply NOT_FOUND otherwise.
- **Conversation memory (new):** the last answered turns are supplied to the model and used to rewrite follow-ups such as "and can it be condoned?" into a standalone question. With a model the rewrite is done by the model; without one, a rule detects short or pronoun-led follow-ups and prepends the previous question. The conversation is stored per user and restored at the next login; a "Start new conversation" button inserts a reset marker so no history table is altered.
- **Offline mode:** without a key the assistant shows the best-matching excerpts instead of a written answer.

### *7.2 Snap-and-Explain*

A student uploads or photographs a textbook page, circuit, graph or handwritten problem, chooses English, Telugu or both, and a level. Gemma 4 vision returns a topic, a step-by-step explanation, key formulas, common mistakes and three practice questions with hidden answers. Small images trigger a retake warning, and a notice asks students to verify with their faculty.

### *7.3 Diagram Reviewer*

The model converts the image into a structure (nodes and edges). Deterministic rules then judge it, so scoring is consistent: flowcharts need Start and End terminators, decisions need at least two labelled branches, and there must be no dead ends or unreachable shapes; ER diagrams need attributes, a key per entity, at least two entities per relationship and cardinality labels; UML class diagrams need class names and no inheritance cycles; all types check dangling arrows and orphan shapes. Score = 100 - 20 x errors - 5 x warnings. The structure is drawn as a Graphviz chart, and a JSON tab allows the rule checks to run without any model.

### *7.4 Answer-Script Feedback Assistant*

Faculty choose a rubric (YAML, validated so point marks sum to the maximum), then upload scanned answers or paste typed text. The model transcribes the answer and decides covered, partial or missing for each rubric point with an evidence quote and confidence. Code clamps every suggested mark to the point's limit, fills missing points with zero, and flags confidence below 0.6. Faculty edit marks in a table and press approve; only approved marks are stored and exported to CSV. Every suggestion and approval is audited.

### *7.5 CO-PO Mapper and Question Paper Auditor*

- **Extraction:** questions and marks are extracted by the model, or by a regular-expression parser when no model is available.

- **Tagging:** each question receives a course outcome and a Bloom's level, by the model or by rules (Bloom verb lexicon, BM25 overlap with CO text).
- **Coverage:** marks per CO and per Bloom level are compared with targets in the outcomes file, and warnings are raised for uncovered COs or over- and under-weighted levels.
- **PO weights:** PO weight = sum over COs of (CO share of marks x mapping level / 3).
- **Repeat detection:** questions are compared with stored past papers using cosine similarity of at least 0.88 with embeddings, or Jaccard similarity of at least 0.6 without.
- **Export:** Excel workbook (questions, CO coverage, Bloom, PO weights, repeats) and CSV.

### *7.6 Timetable Generator with labs (new)*

**Inputs from the interface:** working days, periods per day, first start time, period length, up to two breaks, classrooms, lab rooms, a course table (course, name, faculty, section, theory hours, lab hours, lab block length, optional lab assistant) and faculty unavailable slots.

**Hard constraints:** no faculty, lab assistant, section or room is double-booked; labs run in lab rooms as consecutive periods of the chosen block length and never cross a break; unavailable slots are respected; at most two theory sessions of one course per day and one lab block per course per day.

**Soft preferences (penalties):** spread a course over the week, avoid idle gaps for sections, keep faculty at five hours a day or fewer, place labs in the afternoon and theory in the morning, and keep a section in one classroom.

**Algorithm.** Sessions are generated from the course table, ordered labs first and longest first, and placed one at a time at the (day, start, room) candidate with the lowest penalty, with small random tie-breaking. The search restarts with different seeds (default 25) and keeps the result with the fewest unplaced sessions, then the lowest penalty. Sessions that cannot be placed are listed with the dominant blocking reason instead of being hidden. An independent validator re-checks every hard constraint on the finished timetable and its result is shown on screen.

**Outputs and sharing.** Views by section, faculty and room, and a flat session list. Downloads: Excel (one sheet per view), printable HTML and CSV. A public, read-only link of the form `https://<app>/?tt=CODE` is stored in the database; the owner can remove it. All user text is escaped in exported HTML.

### *7.7 Career Roadmap*

A course catalogue (credits, prerequisites, skills) and a goal file (required skills with importance weights, self-study hints, projects, certificates) drive the planner. The student picks a goal, ticks completed courses, and sets a credit cap and number of semesters.

- **Selection:** greedy weighted coverage. At each step the planner adds the course with the highest ratio of newly covered skill weight to credits, counting any missing prerequisites in the cost, until no course adds coverage.
- **Scheduling:** semesters are filled with courses whose prerequisites were completed in an earlier semester, preferring courses that unlock the most others, within the credit cap.
- **Output:** skill coverage percentage, credits, semester tables, a prerequisite graph (green planned, grey completed), skills not covered by any course with self-study hints, and a Markdown download. An optional model call explains the plan in plain words.

### *7.8 Interview Mock Bot*

The student chooses viva or placement practice, a topic and a number of questions from a YAML question bank. Each question is read aloud by the browser. The student records an answer with the microphone, the audio is transcribed by an audio-capable model, and the transcript can be corrected or typed instead. Each answer is scored on a 0 to 10 scale for correctness and clarity.

- **Correctness (rules):** each key point has keywords; a point is covered when enough keywords occur in the answer, and the score is the mean coverage x 10.
- **Clarity (rules):** starts at 10 and loses points for very short or very long answers, filler words per 100 words, overlong or choppy sentences, repeated words and, when audio duration is known, speaking pace outside 110 to 170 words per minute. Each deduction produces a specific tip.
- **With a model:** a strict interviewer prompt returns correctness, clarity, covered and missing points, mistakes, feedback and a model answer. Final correctness = 0.7 x model + 0.3 x rules; final clarity = 0.5 x model + 0.5 x rules, so measurable delivery habits still count.

$$Overall = 0.6 \times Correctness + 0.4 \times Clarity \qquad (1)$$

Every scored answer is stored, and a progress page shows averages, a trend chart and weakest topics. Without a key the student still records and then types the answer, and rule-based scoring applies.

### *7.9 Placement Probability*

The module estimates the chance of being shortlisted if the student applies to a company. It combines the company's historical shortlisting rate with the student's profile.

$$\theta \sim Beta(a_0 + s,\ b_0 + n - s), \quad a_0 = k r,\ \ b_0 = k(1 - r) \qquad (2)$$

$$\text{logit}(p) = \text{logit}(\theta) + \sum_i w_i (x_i - \text{ref}_i) - 0.8 \times \text{backlogs} \qquad (3)$$

In (2), s of n past applicants were shortlisted, and the prior has rate r = 0.15 and strength k = 4, so a company with little history yields a wide posterior. In (3), x_i are the student's features (weighted skill match, CGPA, coding practice, projects, internship, communication) and ref_i the company's typical applicant. The weights w_i are 3.0, 0.9, 1.5, 1.0, 0.8 and 0.6 respectively. Four thousand Monte Carlo draws of theta give the mean and a 90 percent credible interval. Eligibility rules (minimum CGPA, maximum backlogs) are a hard gate that returns zero with the reason.

**What to improve.** The module re-estimates the probability under single changes (learn each missing required skill, 100 more practice problems, one more project, an internship, +0.3 CGPA, better communication, clearing a backlog) using the same random draws, and ranks them by the gain in percentage points. The weights are expert-set, not fitted to real outcomes, and the bundled company history is fictional; the placement cell should load real history before the numbers are used with students.

### *7.10 Peer Project Showcase*

- **Upload and auto-tagging:** title, description, link and an optional README. Tags come from a technology dictionary and domain keywords (word-boundary matching), merged with model-extracted tech stack, domains, difficulty and a one-line summary when a model is available. Users can edit the tags before publishing.
- **Similar projects:** similarity = 0.5 x text similarity (embedding cosine, or TF-IDF cosine offline) + 0.5 x tag similarity, where tag similarity = 0.6 x Jaccard(tech) + 0.4 x Jaccard(domains). A publish-time notice lists close existing projects.
- **Teammates:** the gap is the set of technologies used by similar projects but absent from the owner's project and skills. A candidate scores 0.6 x (gap skills they know / gap size) + 0.4 x domain overlap. Only people who ticked "open to teaming" in their profile appear.
- **Permissions:** owners delete their own projects; administrators can delete any.

### *7.11 Administrator modules*

- **Knowledge Base Manager:** multi-file upload with category, audience, version and effective date; archive, activate, delete, re-index embeddings, and a search tester that shows scores as a chosen role.

- **Config Manager:** edit any YAML with validation; faculty have read-only access; files can be downloaded to commit to the repository.
- **User Management:** add users, bulk import from CSV, activate or deactivate, reset passwords.
- **Usage, Audit and Health:** counts, database and model status, test buttons for the model and embeddings, usage by module, answer feedback, unanswered questions (content gaps) and the audit log with CSV export.

## 8. Data Model

**TABLE IV**
DATABASE TABLES

| Table | Main fields |
|---|---|
| users | username, name, role, password_hash, active |
| documents | title, category, audience, version, effective_date, status, filename |
| chunks | document_id, page, text, embedding (JSON) |
| chat_messages | username, question, answer, sources, found, feedback |
| chat_resets | username, up_to_id (conversation restart marker) |
| evaluations | faculty, course, question_id, student_ref, result_json, final_marks, status |
| paper_audits, past_questions | questions and coverage JSON; past questions with optional vectors |
| timetables | share_code, owner, title, payload (JSON) |
| interview_sessions | username, category, question, transcript, correctness, clarity, overall |
| profiles | username, skills, interests, open_to_team |
| projects | owner, title, description, link, tech, domains, difficulty, summary, embedding |
| audit_log | username, role, module, action, model, time |

## 9. Configuration Reference

**TABLE V**
ENVIRONMENT VARIABLES AND SECRETS

| Name | Default | Meaning |
|---|---|---|
| GEMINI_API_KEY | none | Enables Gemma 4, embeddings and transcription through the Gemini API |
| LLM_BACKEND | gemini if a key exists, else none | gemini, ollama or none |
| GEMMA_MODEL | gemma-4-26b-a4b-it | Text and vision model (alternative: gemma-4-31b-it) |
| EMBED_MODEL | gemini-embedding-001 | Embedding model |
| TRANSCRIBE_MODEL | gemini-2.5-flash | Audio-capable model for voice answers; set to any model your key supports |
| DATABASE_URL | sqlite:///data/campus.db | Use a PostgreSQL URL for persistence |
| OLLAMA_URL, OLLAMA_MODEL, OLLAMA_EMBED_MODEL | localhost defaults | Local backend settings |

**URL parameters.** `?tt=CODE` opens a shared timetable without login.

## 10. Security and Privacy

- Role checks happen in code, menus are built from the role, and retrieval filters by audience inside the query.
- Passwords use salted PBKDF2; there is no single sign-on yet.
- With the Gemini API, prompts, retrieved passages, images and audio are sent to Google. Do not upload real student scripts or personal data to a public demo; use Ollama for sensitive material.
- Uploaded document text is fenced in the prompt and the model is told to ignore instructions inside it. This reduces prompt-injection risk but does not remove it.
- AI marks are clamped to the rubric and saved only after faculty approval; AI actions are audited.
- Shared timetable links are public to anyone who has the code; they can be removed by the owner. Exported HTML escapes user text.
- Teammate suggestions use only people who opted in. Project and interview data are tied to the user's account.
- Secrets are kept in environment variables or the platform secret store, never in the repository.

## 11. Deployment

1. Push the repository to GitHub (the `.env` and `secrets.toml` files are git-ignored).
2. On Streamlit Community Cloud choose Create app, select the repository, branch `main` and main file `app.py`, and set Python 3.11.
3. Under Advanced settings, Secrets, add `GEMINI_API_KEY` and, for persistence, `DATABASE_URL`.
4. Deploy, open the URL in a private window, sign in as each demo role, and run the live checks in Section 13.
5. Open the URL shortly before any demonstration, because free apps sleep after inactivity.

**TABLE VI**
DATABASE OPTIONS

| Option | Setup | Persistent on Streamlit Cloud | Use for |
|---|---|---|---|
| SQLite (default) | None | No, resets on restart | Local work, quick demos |
| Neon PostgreSQL (free tier) | Create project, copy URL with sslmode=require | Yes | Live demo with uploaded data |
| Supabase PostgreSQL (free tier) | Copy the session pooler URL | Yes | Alternative to Neon |

**Local and Docker.** `streamlit run app.py` for local use; `docker compose up --build` mounts the data folder so SQLite survives restarts. Ollama can supply the model and embeddings for a fully local run.

## Appendix A. Bundled Test and Sample Data

**TABLE XII**
FILES USED FOR TESTING

| Location | Content |
|---|---|
| data/sample_docs/ | Eight fictional university documents in TXT, MD, CSV, DOCX and PDF; one faculty-only document tests role filtering |

| Location | Content |
|---|---|
| data/test_data/rag_test_questions.csv | 21 questions with roles, expected keywords, out-of-scope cases and a Telugu case |
| data/test_data/answer_scripts/ | Good, partial and poor answers to the stack question as images and text; a binary search answer |
| data/test_data/question_papers/ | Sample paper and a previous paper with two repeated questions |
| data/test_data/diagrams/ | Correct and broken flowchart, ER diagram, as images and structure JSON |
| data/test_data/snap/ | Textbook page, RC circuit and parabola graph images |
| data/test_data/projects_sample.json | Six fictional projects and five profiles for similarity and teammate demos |
| data/test_data/users/ | CSV for bulk user import |